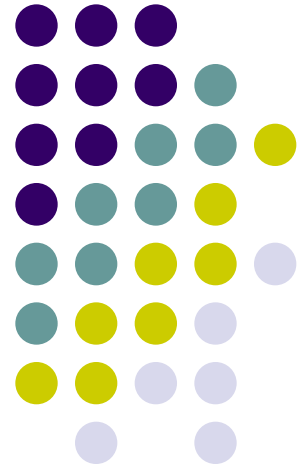


php



Dr:Wafaa Alnbhany

Lecture6



PHP Functions

The real power of PHP comes from its functions.

In PHP - there are more than 700 built-in functions available.

PHP Functions

In this tutorial we will show you how to create your own functions.

For a reference and examples of the built-in functions, please visit our [PHP Reference](#).

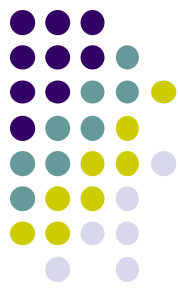
Create a PHP Function

A function is a block of code that can be executed whenever we need it.

Creating PHP functions:

- All functions start with the word "function()"
- Name the function - It should be possible to understand what the function does by its name. The name can start with a letter or underscore (not a number)
- Add a "{" - The function code starts after the opening curly brace
- Insert the function code
- Add a "}" - The function is finished by a closing curly brace

Example



A simple function that writes my name when it is called:

```
<html>
<body>
<?php
function writeMyName()
{
    echo "Kai Jim Refsnes";
}
writeMyName();
?>
</body>
</html>
```

Use a PHP Function

Now we will use the function in a PHP script:

```
<html>
<body>
<?php
function writeMyName()
{
    echo "Kai Jim Refsnes";
}
echo "Hello world!<br />";
echo "My name is ";
writeMyName();
echo "<br />That's right, ";
writeMyName();
echo " is my name.";
?>
</body>
</html>
```

The output of the code above will be:

```
Hello world!
My name is Kai Jim Refsnes.
That's right, Kai Jim Refsnes is my name.
```

PHP Functions - Adding parameters

Our first function (`writeMyName()`) is a very simple function. It only writes a static string.

To add more functionality to a function, we can add parameters. A parameter is just like a variable.

You may have noticed the parentheses after the function name, like: `writeMyName()`. The parameters are specified inside the parentheses.

Example 1

The following example will write different first names, but the same last name:

```
<html>
<body>
<?php
function writeMyName($fname)
{
    echo $fname . " Refsnes.<br />";
}
echo "My name is ";
writeMyName("Kai Jim");
echo "My name is ";
writeMyName("Hege");
echo "My name is ";
writeMyName("Stale");
?>
</body>
</html>
```

The output of the code above will be:

```
My name is Kai Jim Refsnes.
My name is Hege Refsnes.
My name is Stale Refsnes.
```

Example 2

The following function has two parameters:

```
<html>
<body>
<?php
function writeMyName($fname,$punctuation)
{
    echo $fname . " Refsnes" . $punctuation . "<br />";
}
echo "My name is ";
writeMyName("Kai Jim",".");
echo "My name is ";
writeMyName("Hege","!");
echo "My name is ";
writeMyName("Ståle","...");
?>
</body>
</html>
```

The output of the code above will be:

```
My name is Kai Jim Refsnes.  
My name is Hege Refsnes!  
My name is Ståle Refsnes...
```

PHP Functions - Return values

Functions can also be used to return values.

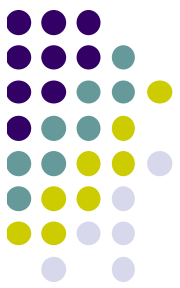
Example

```
<html>  
<body>  
<?php  
function add($x,$y)  
{  
    $total = $x + $y;  
    return $total;  
}  
echo "1 + 16 = " . add(1,16);  
?>  
</body>  
</html>
```

The output of the code above will be:

```
1 + 16 = 17
```


PHP Forms and User Input



The PHP \$_GET and \$_POST variables are used to retrieve information from forms, like user input.

PHP Form Handling

The most important thing to notice when dealing with HTML forms and PHP is that any form element in an HTML page will **automatically** be available to your PHP scripts.

```
<html>
<body>
<form action="welcome.php" method="post">
Name: <input type="text" name="name" />
Age: <input type="text" name="age" />
<input type="submit" />
</form>
</body>
</html>
```

The example HTML page above contains two input fields and a submit button. When the user fills in this form and click on the submit button, the form data is sent to the "welcome.php" file.

The "welcome.php" file looks like this:

```
<html>
<body>
Welcome <?php echo $_POST["name"]; ?>.<br />
You are <?php echo $_POST["age"]; ?> years old.
</body>
</html>
```

A sample output of the above script may be:

```
Welcome John.
You are 28 years old.
```

Form Validation

User input should be validated whenever possible. Client side validation is faster, and will reduce server load.

However, any site that gets enough traffic to worry about server resources, may also need to worry about site security. You should always use server side validation if the form accesses a database.

A good way to validate a form on the server is to post the form to itself, instead of jumping to a different page. The user will then get the error messages on the same page as the form. This makes it easier to discover the error.

PHP \$_GET

The \$_GET variable is used to collect values from a form with method="get".

The \$_GET Variable

The \$_GET variable is an array of variable names and values sent by the HTTP GET method.

The \$_GET variable is used to collect values from a form with method="get". Information sent from a form with the GET method is visible to everyone (it will be displayed in the browser's address bar) and it has limits on the amount of information to send (max. 100 characters).

Example

```
<form action="welcome.php" method="get">
Name: <input type="text" name="name" />
Age: <input type="text" name="age" />
<input type="submit" />
</form>
```

When the user clicks the "Submit" button, the URL sent could look something like this:

```
http://www.w3schools.com/welcome.php?name=Peter&age=37
```

The "welcome.php" file can now use the `$_GET` variable to catch the form data (notice that the names of the form fields will automatically be the ID keys in the `$_GET` array):

```
Welcome <?php echo $_GET["name"]; ?>.<br />
You are <?php echo $_GET["age"]; ?> years old!
```

Why use \$_GET?

Note: When using the \$_GET variable all variable names and values are displayed in the URL. So this method should not be used when sending passwords or other sensitive information! However, because the variables are displayed in the URL, it is possible to bookmark the page. This can be useful in some cases.

Note: The HTTP GET method is not suitable on large variable values; the value cannot exceed 100 characters.

The \$_REQUEST Variable

The PHP \$_REQUEST variable contains the contents of both \$_GET, \$_POST, and \$_COOKIE.

The PHP \$_REQUEST variable can be used to get the result from form data sent with both the GET and POST methods.

Example

```
Welcome <?php echo $_REQUEST["name"]; ?>.<br />
You are <?php echo $_REQUEST["age"]; ?> years old!
```

PHP \$_POST

The \$_POST variable is used to collect values from a form with method="post".

The \$_POST Variable

The \$_POST variable is an array of variable names and values sent by the HTTP POST method.

The \$_POST variable is used to collect values from a form with method="post". Information sent from a form with the POST method is invisible to others and has no limits on the amount of information to send.

Example

```
<form action="welcome.php" method="post">
Enter your name: <input type="text" name="name" />
Enter your age: <input type="text" name="age" />
<input type="submit" />
</form>
```

When the user clicks the "Submit" button, the URL will not contain any form data, and will look something like this:

```
http://www.w3schools.com/welcome.php
```

The "welcome.php" file can now use the `$_POST` variable to catch the form data (notice that the names of the form fields will automatically be the ID keys in the `$_POST` array):

```
Welcome <?php echo $_POST["name"]; ?>.<br />
You are <?php echo $_POST["age"]; ?> years old!
```

Why use `$_POST`?

- Variables sent with HTTP POST are not shown in the URL
- Variables have no length limit

However, because the variables are not displayed in the URL, it is not possible to bookmark the page.

The \$_REQUEST Variable

The PHP \$_REQUEST variable contains the contents of both \$_GET, \$_POST, and \$_COOKIE.

The PHP \$_REQUEST variable can be used to get the result from form data sent with both the GET and POST methods.

Example

```
Welcome <?php echo $_REQUEST["name"]; ?>.<br />
You are <?php echo $_REQUEST["age"]; ?> years old!
```

PHP Include File

Server Side Includes (SSI) are used to create functions, headers, footers, or elements that will be reused on multiple pages.

Server Side Includes

You can insert the content of a file into a PHP file before the server executes it, with the `include()` or `require()` function. The two functions are identical in every way, except how they handle errors. The `include()` function generates a warning (but the script will continue execution) while the `require()` function generates a fatal error (and the script execution will stop after the error).

These two functions are used to create functions, headers, footers, or elements that can be reused on multiple pages.

This can save the developer a considerable amount of time. This means that you can create a standard header or menu file that you want all your web pages to include. When the header needs to be updated, you can only update this one include file, or when you add a new page to your site, you can simply change the menu file (instead of updating the links on all web pages).

The include() Function

The include() function takes all the text in a specified file and copies it into the file that uses the include function.

Example 1

Assume that you have a standard header file, called "header.php". To include the header file in a page, use the include() function, like this:

```
<html>
<body>
<?php include("header.php"); ?>
<h1>Welcome to my home page</h1>
<p>Some text</p>
</body>
</html>
```

Example 2

Now, let's assume we have a standard menu file that should be used on all pages (include files usually have a ".php" extension). Look at the "menu.php" file below:

```
<html>
<body>
<a href="http://www.w3schools.com/default.php">Home</a> |
<a href="http://www.w3schools.com/about.php">About Us</a> |
<a href="http://www.w3schools.com/contact.php">Contact Us</a>
```

The three files, "default.php", "about.php", and "contact.php" should all include the "menu.php" file. Here is the code in "default.php":

```
<?php include("menu.php"); ?>
<h1>Welcome to my home page</h1>
<p>Some text</p>
</body>
</html>
```

If you look at the source code of the "default.php" in a browser, it will look something like this:

```
<html>
<body>
<a href="default.php">Home</a> |
<a href="about.php">About Us</a> |
<a href="contact.php">Contact Us</a>
<h1>Welcome to my home page</h1>
<p>Some text</p>
</body>
</html>
```

And, of course, we would have to do the same thing for "about.php" and "contact.php". By using include files, you simply have to update the text in the "menu.php" file if you decide to rename or change the order of the links or add another web page to the site.

The require() Function

The require() function is identical to include(), except that it handles errors differently.

The include() function generates a warning (but the script will continue execution) while the require() function generates a fatal error (and the script execution will stop after the error).

If you include a file with the include() function and an error occurs, you might get an error message like the one below.

PHP code:

```
<html>
<body>

<?php
include("wrongFile.php");
echo "Hello World!";
?>

</body>
</html>
```

Error message:

```
Warning: include(wrongFile.php) [function.include]:  
failed to open stream:
```

```
No such file or directory in C:\home\website\test.php on line  
5
```

```
Warning: include() [function.include]:  
Failed opening 'wrongFile.php' for inclusion  
(include_path='.;C:\php5\pear')  
in C:\home\website\test.php on line 5  
Hello World!
```

Notice that the echo statement is still executed! This is because a Warning does not stop the script execution.

Now, let's run the same example with the require() function.

PHP code:

```
<html>
<body>

<?php
require("wrongFile.php");
echo "Hello World!";
?>

</body>
</html>
```

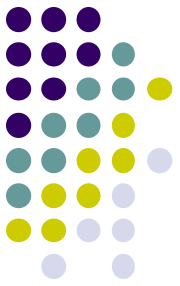
Error message:

```
Warning: require(wrongFile.php) [function.require]:
failed to open stream:
No such file or directory in C:\home\website\test.php on line
5
Fatal error: require() [function.require]:
Failed opening required 'wrongFile.php'
(include_path='.;C:\php5\pear')
in C:\home\website\test.php on line 5
```

The echo statement was not executed because the script execution stopped after the fatal error.

It is recommended to use the `require()` function instead of `include()`, because scripts should not continue executing if files are missing or misnamed.

H.W



- What are difference between post and get
- What are difference between include and require